Mini Project E534: Data Analytics Pipeline with AWS and PySpark

Connor Oman

Introduction:

Through this class and this final project, I have learned a lot about the intricacies and challenges that come with big data. Therefore, my top priority for this assignment was to find a dataset that encapsulated the idea of "big data". I spent a day combing through public websites and sources like Kaggle, U.S. census data, and Julian McCauley's repository for review data and recommender systems. I decided not to pursue a dataset from the U.S. census data as most datasets were rather unclean with a higher scope than I would've liked for this project and I did not use any of Julian McCauley's data because I found it to be highly tailored towards engineering recommender systems. Thus, I primarily shifted my focus to looking for public datasets in Kaggle with some key constraints in mind. First, the dataset must be at least one million rows for me to be confident it qualifies as big data. Second, the dataset must contain a mix of locational, temporal, and categorical data with at least 1 relevant numerical column to draw conclusions from. Finally, I wanted the dataset to be related to financial transactions of some kind because I felt like the wording and examples from the assignment were tailored to that type of data. With these constraints, I struggled to find a dataset that was large enough, but eventually I found an exemplary dataset on Kaggle that works for this project.

The dataset is called: "Online Store Customer Transactions" (Chalise, 2025) and it was created by a normal user named Noro Chalise. This dataset was uploaded to Kaggle in 2023 and was last updated 9 months ago and is essentially a generic online transaction log. It contains 1 million rows (which satisfied my first constraint) of anonymized transaction IDs across the time period of 2018 to 2025 with demographic data contextualizing the purchase like age and employment status. In the dataset, there are 11 columns: Transaction_date, Transaction_ID, Gender, Age, Martial_status, State_names, Segment, Employee_status, Payment_method, Referral, and Amount_spent. I'll briefly explain some of the less obvious columns. Segment refers to the customer's membership with the company broken into tiers like "Basic", "Silver", and "Platinum" which receive different benefits. Payment method can only be 1 of three things, "Paypal", "Card", and "Other". Referral is a boolean column (with values only being 0 or 1) and

refers to whether the customer used a referral link when making the purchase. Finally, Amount_spent is the most relevant column for analysis as it's a numerical column that dictates how much was spent in the purchase. From this column, I was able to derive key metrics from the dataset like what state has generated the most total money or how revenue trends change month to month or year to year.

However, despite Online Store Customer Transactions being perfect for the purpose of this assignment, it is technically a synthetic dataset. However, as the assignment doesn't specify that the data must be from a real company, I felt the usage of this synthetic dataset would be equivalent to a real dataset for the purpose of this project as the assignment is primarily focused on the AWS to Pyspark pipeline. I found some financial transaction datasets that were from real sources on Kaggle, but they rarely met the 1 million row requirement. Therefore, I felt justified in proceeding with this dataset, but I still made sure to check how the dataset was generated beforehand. Noro Chalise derived this new dataset from the Kaggle online store dummy dataset (which is a dataset commonly used for learning) and expanded upon the data to reach 1 million rows to better serve machine learning projects. Furthermore, on Kaggle, you can preview the distribution of values in numerical columns and the distribution of Age and Amount_spent were very balanced and not skewed/biased towards any specific ages or expenditures. Jumping ahead a bit, in the methodology section, I checked the dataset for outliers and found no points outside of the 3rd standard deviation, indicating the data was already fairly clean and balanced before being processed. Therefore, I felt confident that despite the synthetic nature of the dataset, Online Store Customer Transactions would be more than enough to enhance my AWS skills and draw meaningful conclusions. Going even further, the importance of ethics in respect to big data is a core tenant in what we've learned from class. When I discuss ethical concerns in the project's conclusion, I will treat this synthetic dataset as if it were real for the sake of the assignment.

Now with the dataset in place, I wanted to learn about the revenue and how age impacted website traffic. Thus I set forth with several key questions in mind. How does revenue change over time? Which states generate the most and least revenue? Which age bracket is the most common? Through designing an AWS pipeline, processing data and deriving insights in Pyspark,

generating visualizations in AWS quick insight, and writing this paper, I seek to answer these questions. From this project, I hope to learn more about the infrastructure of AWS and how to efficiently handle and debug any problem that may arise.

Methodology:

I closely followed the project workflow provided in the Mini Project - E534.pdf document throughout the project. Therefore, the methodology section will be ordered chronologically and will cover sections two (Environment Setup) through four (Visualization), but the figures will be saved for the results section. The methodology will recap what I did for each step and recount any bugs or issues I ran into during implementation.

Section two or "Environment Setup" overall went rather smoothly. I was able to sign-up for an AWS free account, set up AWS CLI, and set up a new IAM user called "ConnorIU" so I wasn't exclusively operating in the root user account. Then I set up a S3 bucket with the dataset and created an EC2 instance all with zero issues. After some deliberation, I decided to run the EC2 linux instance on my local machine instead of the web browser version because I felt like it would be faster in the long run because I could easily reference files from my own computer.

```
C:\Users\Conno>aws configure
AWS Access Key ID [****************D2EY]: AKIAY5C
AWS Secret Access Key [****************rgiT]: 864
Default region name [us-west-2]:
Default output format [json]:
```

After some minor user interface navigation troubles, I was able to locate the public IPv4 address for the instance. AWS also recommended that I set up a remote access key as well for security reasons, so I created a .ssh folder in C:\Users\<my username> and downloaded the .pem file to that location. Then, with the correct file path to the remote access key and the public IPv4 address, I was able to remotely connect to the EC2 linux instance from my own computer.

```
C:\Users\Conno>ssh -i C:\Users\Conno\.ssh\hippo81632.pem ec2-user@18.224.59.68
The authenticity of host '18.224.59.68 (18.224.59.68)' can't be established.
ED25519 key fingerprint is SHA256:wS8S+rC84phyV8GrIRCaDaxkLdHdXg1P461+kpJrT1k.
This host key is known by the following other names/addresses:
    C:\Users\Conno/.ssh/known_hosts:3: 18.217.38.105
    C:\Users\Conno/.ssh/known_hosts:5: 3.22.250.121
    C:\Users\Conno/.ssh/known_hosts:6: 18.220.63.49
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.224.59.68' (ED25519) to the list of known hosts.

A newer release of "Amazon Linux" is available.
  Version 2023.9.20251208:
Run "/usr/bin/dnf check-release-update" for full release and version update info
```

However, the first issues began to arise when I tried to initialize Pyspark.

In section 2.2 step 1, I was able to install Java 17 no problem, but I had trouble initializing Pyspark because I had not set up JAVA_HOME properly. Prior to this assignment, I had no idea what JAVA_HOME was or why it was relevant to Pyspark. Before I found the issue, when I installed Pyspark, it wouldn't immediately output an error message making it appear like it worked properly. When I would try to reference Pyspark later, it would say: "No module named 'pyspark'". After doing some research on why Pyspark couldn't be found, I learned about JAVA_HOME which is an environment variable that routes to where Java was installed. Essentially, Java 17 was installed correctly, but Pyspark had no idea where, so to fix this I had to export JAVA_HOME which properly defined it.

```
[ec2-user@ip-172-31-3-35 ~]$ sudo dnf update -y
sudo dnf install java-17-amazon-corretto-headless -y
pip3 install pyspark==3.5.1 boto3 pandas

mkdir -p ~/spark_jars && cd ~/spark_jars

wget https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-aws/3.3.3/hadoop-aws-3.3.3.jar
wget https://repo1.maven.org/maven2/com/amazonaws/aws-java-sdk-bundle/1.12.367/aws-java-sdk-bundle-1.12.367.jar

echo 'export SPARK_DIST_CLASSPATH=/home/ec2-user/spark_jars/*' >> ~/.bashrc
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto' >> ~/.bashrc
source ~/.bashrc
```

In section 2.2 step 3, I ran into a second issue where my local machine could not retrieve any data from the S3 bucket. However, this was a simple fix as I was missing a Hadoop to AWS connector that was required to pull the data from S3 down to my local linux instance. The solution was two wget commands to install the required Hadoop dependencies and then I was able to connect to S3. Aside from the JAVA_HOME debacle, I was able to accomplish everything in section 2 with relative ease.

Moving onto section 3 or “Data Pipeline Tasks”, after installing the required Hadoop dependencies, I was able to retrieve the dataset from S3 and inspect it with df.show(10,truncate =False). From there, the focus became coding Pyspark scripts that could clean, make insights about, and derive new tables from the data.

```
==============================
Row Count: 1000000
Showing first 10 rows:

+--------------+------------------+------+---+--------------+-------------+--------+----------------
------+------------+
|Transaction_ID|Transaction_date   |Gender|Age|Marital_status|State_names   |Segment |Employees_status|
ferral|Amount_spent|
+--------------+------------------+------+---+--------------+-------------+--------+----------------
------+------------+
|1000          |2018-01-01 00:04:00|Female|39 |Single        |Oklahoma      |Platinum|Unemployment    |
      |1557.5      |
|1001          |2018-01-01 00:06:00|Male  |34 |Married       |Hawaii        |Basic   |workers         |
      |153.55      |
|1002          |2018-01-01 00:14:00|Female|53 |Married       |Iowa          |Basic   |self-employed   |
      |2640.96     |
|1003          |2018-01-01 00:23:00|Male  |33 |Married       |Wisconsin     |Basic   |self-employed   |
      |293.58      |
|1004          |2018-01-01 00:25:00|Female|36 |Married       |Texas         |Platinum|Employees       |
```

With respect to cleaning, the first script I made could check numeric columns like Age and Amount_spent in the dataset for outliers. Prior to this assignment, I wasn’t extremely familiar with how to check for outliers or when outliers are significant enough to be removed. Therefore, I did some external research and found a resource about identifying outliers in Spark 3.0 (Tompkins, 2020) that I based some of my script on. I found that it’s best practice to remove any outliers past 3 standard deviations in either direction to limit influence on machine learning models. After designing and executing the script with some minor hiccups (I struggled to find a clean way to display all of the relevant information through print until I found summary tables), I was able to view the relevant information as seen below.

```
+-------------+----+-------+-----------------+-----------------+--------------------+-----------------+
|Column       |Min |Max    |Mean             |StdDev           |3σ_Cutoff (Lower)   |3σ_Cutoff (Upper)|
+-------------+----+-------+-----------------+-----------------+--------------------+-----------------+
|Amount_spent|2.09|2999.98|1416.1261061362104|878.1230821006616|-1218.2431401657743|4050.49535243819 54|
|Age          |15.0|78.0   |46.647169968658034|18.183986041184 79|-7.90478815489633 75|101.1991280922124 |
+-------------+----+-------+-----------------+-----------------+--------------------+-----------------+
```

From this analysis, I found that the minimum and maximum of both Amount_spent and Age did not exceed the cutoffs in either direction meaning there were no outliers significant enough to be required to remove. Therefore, it appears removing outliers is unnecessary for this specific

dataset which further highlights the data's initial cleanliness. I then proceeded with general cleaning steps like dropping NA's from mandatory columns like Transaction_ID, Transaction_date, and Amount_spent. I casted Age to integer and Amount_spent to double where I then derived an additional column from the Amount_spent called total_amount for future analysis. Finally, I extracted the year and month from Transaction_date to new columns year and month.

After the data was sufficiently cleaned, I moved onto the "Data Aggregation" step and I decided to just extend the script from the previous step rather than making a brand new script.

State_Revenue_Ranking

State_names	total_revenue
Illinois	3.5861755640000012E7
Massachusetts	3.385345647000044E7
Missouri	3.3434679229999796E7
Minnesota	3.295374260000035E7
Arizona	3.2220587820000466E7
New Jersey	3.17075109399998 97E7
New York	3.103228385999833E7
Rhode Island	3.04967231400003 06E7
New Mexico	2.961168506999979E7
Kentucky	2.9609199969998777E7
South Dakota	2.9051912329999853E7
Georgia	2.880798610999953E7
Colorado	2.8572518410000615E7
Montana	2.851619387000051E7
West Virginia	2.8378747440000 99E7

only showing top 15 rows

Going back to my key questions, I first wanted to know how the overall total revenue generated from each state compared to each other, so I generated a new table called state_revenue that summed the Amount_spent for every Transaction_ID that was from the same state over the whole time period. By quickly viewing just the top 15 states by revenue I found Illinois had the highest revenue with 3.586E7 closely followed by Massachusetts at 3.385E7 and Missouri at 3.343E7. Next I wanted to see the big picture of how the overall revenue has changed over time. I started by aggregating and summing the total revenue from each year. I found the year that generated the most revenue was 2022 at 1.812E8 dollars and the year that generated the least was 2025 at 1.500E7. However, 2025 being a whole degree of magnitude lower likely indicates that online transaction records for 2025 are incomplete because the year had just started when the dataset was theoretically made. The true lowest year is most likely 2018 at 1.797E8.

Yearly_Revenue_Summary

year	total_revenue
2022	1.8120696208000556E8
2019	1.8115730934001228E8
2023	1.8097881842000654E8
2024	1.8094413477000827E8
2021	1.8084997244999987E8
2020	1.8020432606000093E8
2018	1.7974150237000954E8
2025	1.5002926260000061E7

Monthly_Revenue_Averages

month	avg_revenue
1	1418.578975991457
2	1414.83667900716
3	1419.856451731588
4	1414.398260472217
5	1408.8115912490953
6	1415.2646315673933
7	1421.497017272779
8	1416.3649912279388
9	1414.9662986036853
10	1414.5865979462983
11	1419.2297883122044
12	1414.6657705311804

Next, I wanted to see how the monthly variability compared to the yearly variability. I calculated the average revenue generated per an individual purchase for each month aggregated over the entire period of 2018 to 2025. The monthly average revenues were largely consistent with each other, but July was the only month in the 1420s and May was the only month in the 1400s.

Median_Spend_Per_State

State_names	median_spent
New Jersey	1713.06
Arizona	1710.21
Idaho	1651.44
North Carolina	1651.4
South Dakota	1611.28
Rhode Island	1595.06
New York	1581.77
Montana	1556.63
Missouri	1534.53
Wyoming	1519.41
New Hampshire	1479.67
Nevada	1473.73
Washington	1467.3
Indiana	1465.74
Massachusetts	1455.74

only showing top 15 rows

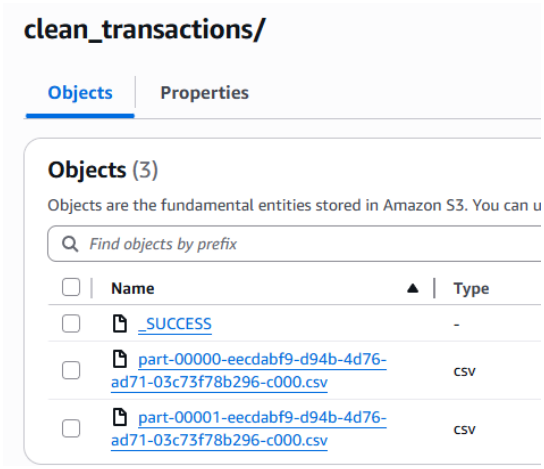
Following that, I felt like it was important to have a table that ranked the states by their average purchase amount to get a general understanding of how your average customer from each state interacts with the store. However, after computing the averages for each state, I realized the Amount_spent column was very unbalanced with some states having a significantly lower number of purchases, but multiple big spenders with purchases above 2750 dollars. To get a better understanding of the normal consumer, I pivoted to calculating the median. Ultimately New Jersey had the highest median Amount_spent per purchase followed very closely by Arizona. Generally, these purchases are much larger than what I'd expect from a real online store, but it's possible that it could be some sort of luxury item or bulk supplier because the dataset is ambiguous. Comparing the median spend per state to the total revenue generated from each state shows New Jersey has a higher proportion of high value purchases than Illinois. Seeing as Illinois isn't even in the top 15, Illinois has more purchases than most states. For the final metric,

I counted the occurrence of different payment methods in the dataset and found that Paypal was the most common payment method at 420534 or a little less than half of the entire dataset.

Payment_Preference

Payment_method	usage_count
PayPal	420534
Card	266328
Other	217073

However, after making these 5 key metrics and brand new aggregation tables, I encountered the next big issue of the project when I tried to upload the dataset back to S3. I made a separate folder to separate the processed data from the raw data, but in my script by mistake I had: root = "s3a://bigdata-connoriu-mini-project/raw" instead of /processed. This caused multiple issues because the extra tables aggregated in section 3.2 went to the wrong folder and there were numerous conflicts between the processed & raw transaction datasets because originally I had them named the exact same. After a bit of trial and error, I fixed the pathing issue, deleted the old files, and for safe future practices I made sure the processed dataset was renamed to clean_transactions. Then finally I was able to see and confirm all the processed tables were properly uploaded and accessible on AWS S3.



Now that the clean datasets were safely stored in S3, I retrieved the cleaned transaction dataset from the processed folder to perform Spark SQL to make 5 different insights. To check my work for the state_revenue aggregate table, I first did a quick query to ensure that the results matched. I selected the State_names column and the SUM() of the Amount_spent column while grouping by State_names and found that the results matched which gave me peace of mind that both methods were working properly as seen below.

SQL PROMPT:

```
Top Revenue States
+-------------+--------------------+
|  State_names|             revenue|
+-------------+--------------------+
|     Illinois| 3.586175564000012E7|
|Massachusetts| 3.385345647000044E7|
|     Missouri|3.3434679229999796E7|
|    Minnesota| 3.295374260000035E7|
|      Arizona|3.2220587820000466E7|
|   New Jersey|3.1707510939999897E7|
|     New York| 3.103228385999833E7|
| Rhode Island|3.0496723140000306E7|
|   New Mexico| 2.961168506999979E7|
|     Kentucky|2.9609199969998777E7|
| South Dakota|2.9051912329999853E7|
|      Georgia| 2.880798610999953E7|
|     Colorado|2.8572518410000615E7|
|      Montana| 2.851619387000051E7|
|West Virginia| 2.8378747440000099E7|
+-------------+--------------------+
```

AGGREGATED TABLE:

```
State_Revenue_Ranking
+-------------+--------------------+
|State_names  |total_revenue       |
+-------------+--------------------+
|Illinois     |3.586175564000012E7 |
|Massachusetts|3.385345647000044E7 |
|Missouri     |3.3434679229999796E7|
|Minnesota    |3.295374260000035E7 |
|Arizona      |3.2220587820000466E7|
|New Jersey   |3.1707510939999897E7|
|New York     |3.103228385999833E7 |
|Rhode Island |3.0496723140000306E7|
|New Mexico   |2.961168506999979E7 |
|Kentucky     |2.9609199969998777E7|
|South Dakota |2.9051912329999853E7|
|Georgia      |2.880798610999953E7 |
|Colorado     |2.8572518410000615E7|
|Montana      |2.851619387000051E7 |
|West Virginia|2.8378747440000099E7|
+-------------+--------------------+
only showing top 15 rows
```

```
Month-Over-Month Revenue
+----+-----+--------------------+
|year|month|     monthly_revenue|
+----+-----+--------------------+
|2018|    1|1.5528042469999978E7|
|2018|    2|1.3850127480000015E7|
|2018|    3|1.5471448829999998E7|
|2018|    4|1.4583011830000004E7|
|2018|    5|1.5324622210000014E7|
|2018|    6|1.4667231820000065E7|
|2018|    7|1.5426546630000062E7|
|2018|    8|1.5246330060000004E7|
|2018|    9|1.4559493820000075E7|
|2018|   10| 1.520623758000004E7|
|2018|   11|1.4792179440000102E7|
|2018|   12|1.5086230200000053E7|
|2019|    1|1.5394457969999991E7|
|2019|    2|1.3808462249999983E7|
|2019|    3|1.5460492760000026E7|
|2019|    4| 1.459343060000001E7|
|2019|    5|1.5437745220000057E7|
|2019|    6| 1.462030679000003E7|
|2019|    7|1.5543384470000016E7|
|2019|    8|1.5492766919999905E7|
|2019|    9|1.4884332049999995E7|
|2019|   10|1.5125519149999939E7|
|2019|   11|1.5212748840000052E7|
|2019|   12|1.5583662320000038E7|
```

As opposed to total yearly revenue, I wanted to see how the revenue varied month to month because it provides more detail in identifying seasonal trends. The query was constructed by ordering by year then month, which splits the months into individual rows for every year allowing us to see a monthly time series. I only returned the first 24 lines of the query just to ensure it worked properly.

```
 Most Common Age Brackets
+---------+----------+
|age_group|user_count|
+---------+----------+
|40-49    |148650    |
|50-59    |146847    |
|30-39    |143766    |
|60-69    |131080    |
|70-79    |128345    |
|20-29    |119778    |
|10-19    |69922     |
|Unknown  |15547     |
+---------+----------+
```

Moving onto the third and most challenging query, I wanted to break the ages up into ten year brackets to see which age was the most common in the transactions dataset. To accomplish this I divided the age by 10, removed the decimal points with floor, and multiplied by 10 to identify the lower bound. Then I simply added by 9 to find the upper bound. All of this is nested in a CONCAT() which defines the range for the COALESCE() function and the COUNT(*) counts the occurrences

of age within the range. The most common age bracket was 40-49 yrs at 148650 entries compared to lower brackets like 20-29 yrs at 119778 and 10-19 yrs at 69922. There were also 15547 entries labeled as “Unknown” which appears to indicate a bug or missing values. However, these values are intentional and I chose to build unknown into my query. Customers could opt out of including their age when making a purchase, so in 15547 transactions, the user chose to not have their age recorded.

For the fourth query, I wanted to see how the employment status affected revenue and the amount of individual transactions. By selecting and grouping by Employee_status, I could count the total number of transactions, total revenue, the average value of a purchase, and the median value of a purchase. The results are as seen below.

Revenue by Employment Status

Employees_status	total_transactions	total_revenue	avg_purchase_value	median_purchase_value
Employees	342531	4.9283858853001 88E8	1438.81455555853	1332.48
workers	285418	3.9671325070000 66E8	1389.9377428893995	1344.94
self-employed	178236	2.5938467119000 4E8	1455.2877712134698	1390.46
Unemployment	87906	1.189151500300 02E8	1352.7535097718244	1190.23
NULL	9844	1.2234291300000392E7	1242.8170763917506	1188.81

As expected, Employees had the most transactions being higher than workers by roughly 60k transactions. The gap in transactions also explains the gap in total revenue. Interestingly, Employees did not have the highest average purchase value with self-employed having both the highest average purchase and median purchase. Similar to “Unknown” in the previous query, NULL just means the Employment_status was not provided with the Transaction_ID. Unlike “Unknown”, it’s unclear what NULL means in this context.

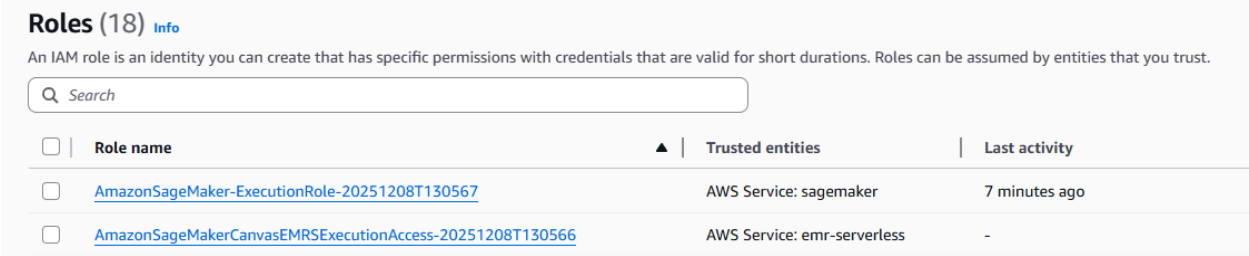
For the final query, I wanted something simpler, but still meaningful. Therefore, I looked at the median Amount_spent per transaction in the state that generated the most revenue: Illinois. I only selected the midpoint of Amount_spent (or percentile_approx(Amount_spent, 0.5)) where the State_names column equaled Illinois which outputs a single line. Illinois’ median Amount_spent per purchase was just

Median Transaction Amount in Illinois

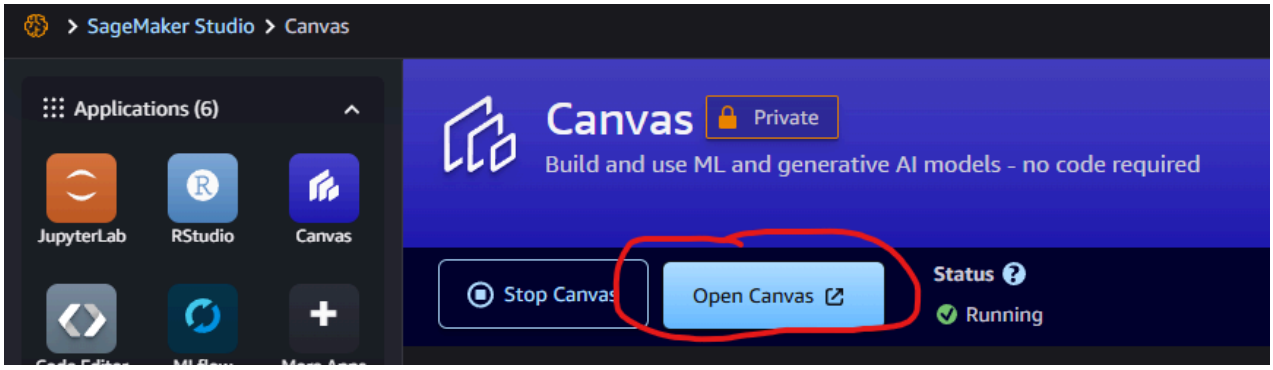
median_spend_IL
1228.67

1228.67 which is lower than Employees, workers, and self-employed from the previous query. This largely confirms what I inferred previously where the size of each purchase is not what makes Illinois so profitable, but rather the volume of purchases.

Moving onto section 3.5, the overall workflow for this section was both challenging and rewarding while showing me the immense strength of SageMaker Autopilot. Navigating to SageMaker studio was a breeze, but I had two small hiccups. First, I struggled to properly set up the permissions for my non-root account because there were very specific permissions I had to grant to the IAM roles.

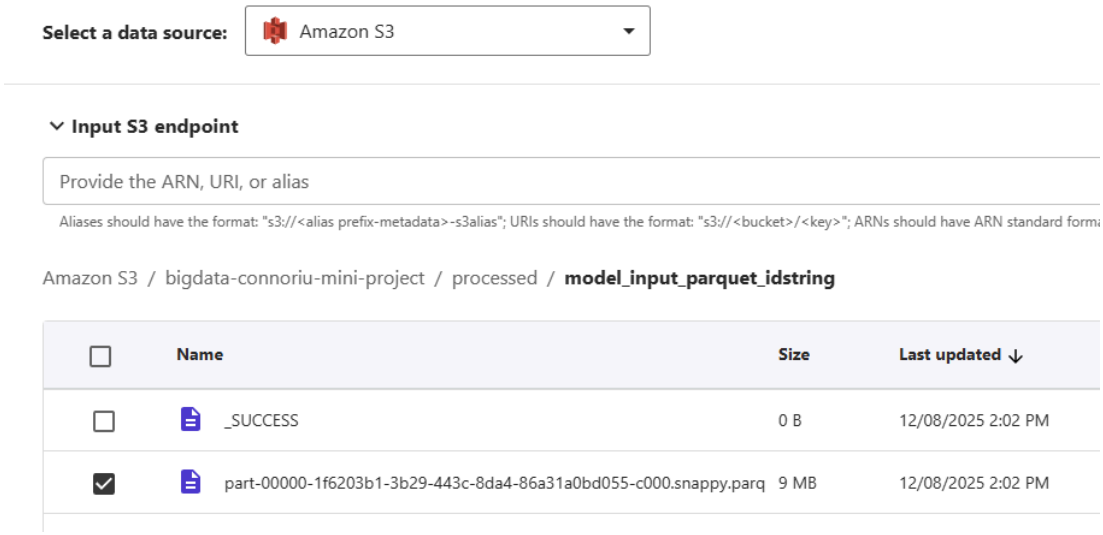


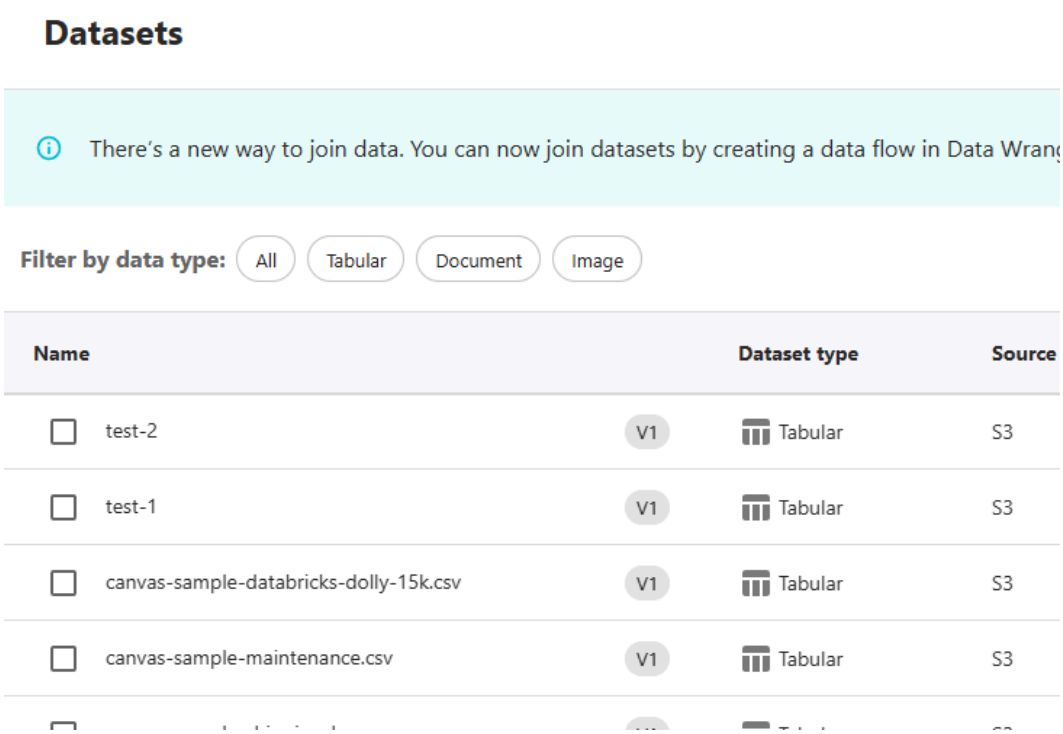
Second, I struggled to figure out what program I should use once inside of SageMaker studio because the first tutorial (the one in the pdf) used a much older interface and the second tutorial from InScribe used SageMaker Classic, but recommended JupyterLab4. After gaining clarification from InScribe, I decided to proceed with Canvas as its usage was permitted and the interface/workflow was the easiest to follow. From there I launched a Canvas instance inside SageMaker studio.



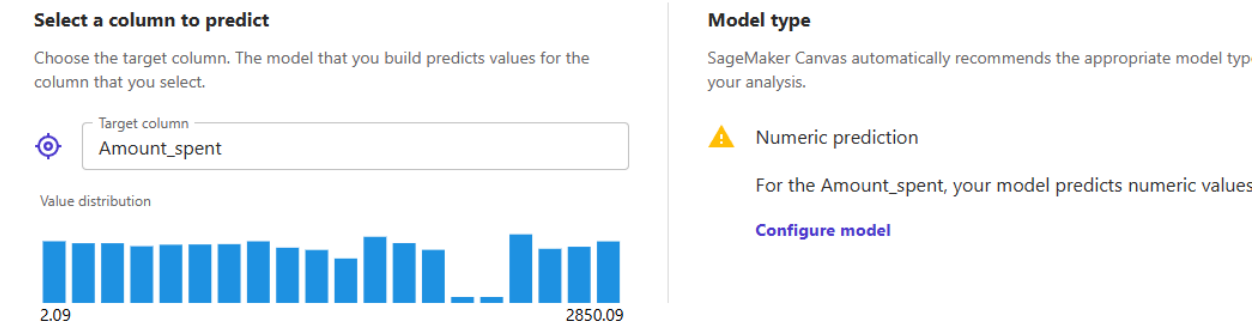
However, after running my first Canvas instance, I immediately ran into another issue. By default, the cleaned dataset that I uploaded back to S3 was actually partitioned into two

different csv files for efficiency and the Canvas data bucket requires a single input. Thus, I wrote a new python script for my local machine that was slightly less efficient, but would ensure all data would be outputted in a singular csv. Thinking I fixed the issue I tried continuing with section 3.5, but I collided with another road block when setting up my model because the singular dataset was a csv file and not a parquet file. I had to go back, change the script to output a single file in parquet format and then I could finally start building the model. I called the dataset using the csv version test-1 and the dataset using the parquet version test-2.





I set up the model by setting Amount_spent as the target column. As I only had 1 relevant numeric column, I decided to design a Numeric prediction model to predict how other columns affect the value of Amount_spent, but first I had to make sure to remove the total_amount column from the model as it was directly derived from Amount_spent.



test-2
Random sample: 20.0k rows

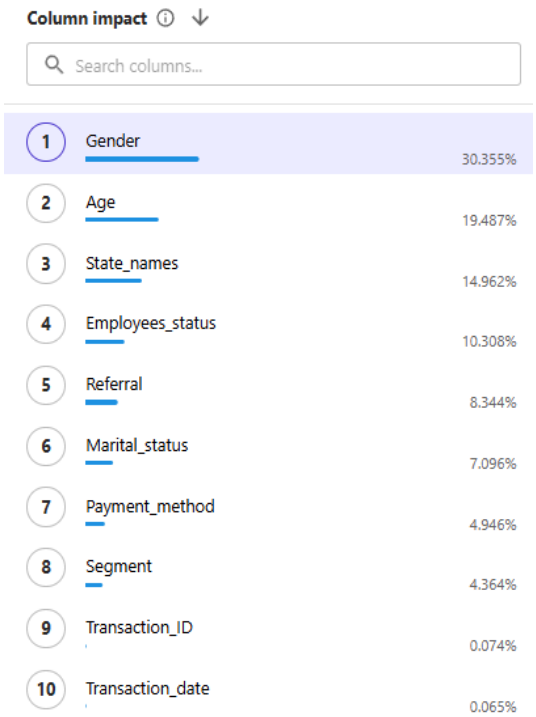
Column name	Data type	Feature type	Missing	Mismatched	Uniqu
year	Numeric	-	0.00% (0)	0.00% (0)	1
Transaction_ID	Numeric	-	0.00% (0)	0.00% (0)	20000
Transaction_date	Datetime	-	0.00% (0)	0.00% (0)	57
total_amount	Numeric	-	0.00% (0)	0.00% (0)	2250

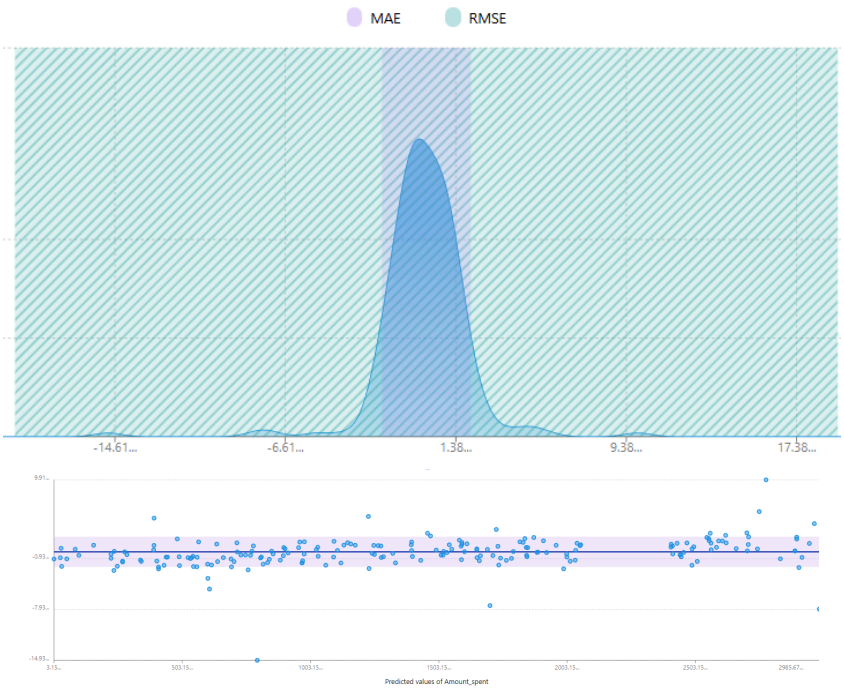
The models took 1 hour and 45 minutes to generate. The best model had a RMSE of 19.308 where the model predicted a value plus or minus 19.31% of the actual value. The R-squared value was 99.95% indicating the regression model explains 99.95% of variability.

Model leaderboard

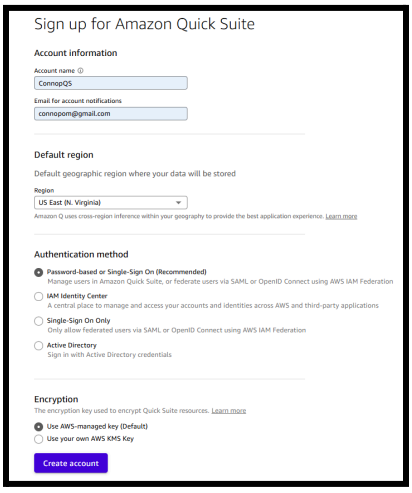
Search leaderboard

Model name	MSE (Optimization)	MAE	RMSE	R2
mqsFvG5wv0KBJ-073-e9186980 (Default model)	372.816	2.082	19.308	99.952%
mqsFvG5wv0KBJ-100-0eb4d598	3420336.750	1627.651	1849.415	-343.572%
mqsFvG5wv0KBJ-099-286bca8d	671.878	1.487	25.921	99.913%
mqsFvG5wv0KBJ-098-6a0b8551	498.864	10.980	22.335	99.935%
mqsFvG5wv0KBJ-097-1f9c01c6	697.815	1.548	26.416	99.909%
mqsFvG5wv0KBJ-096-e88d2240	616.987	2.166	24.839	99.920%





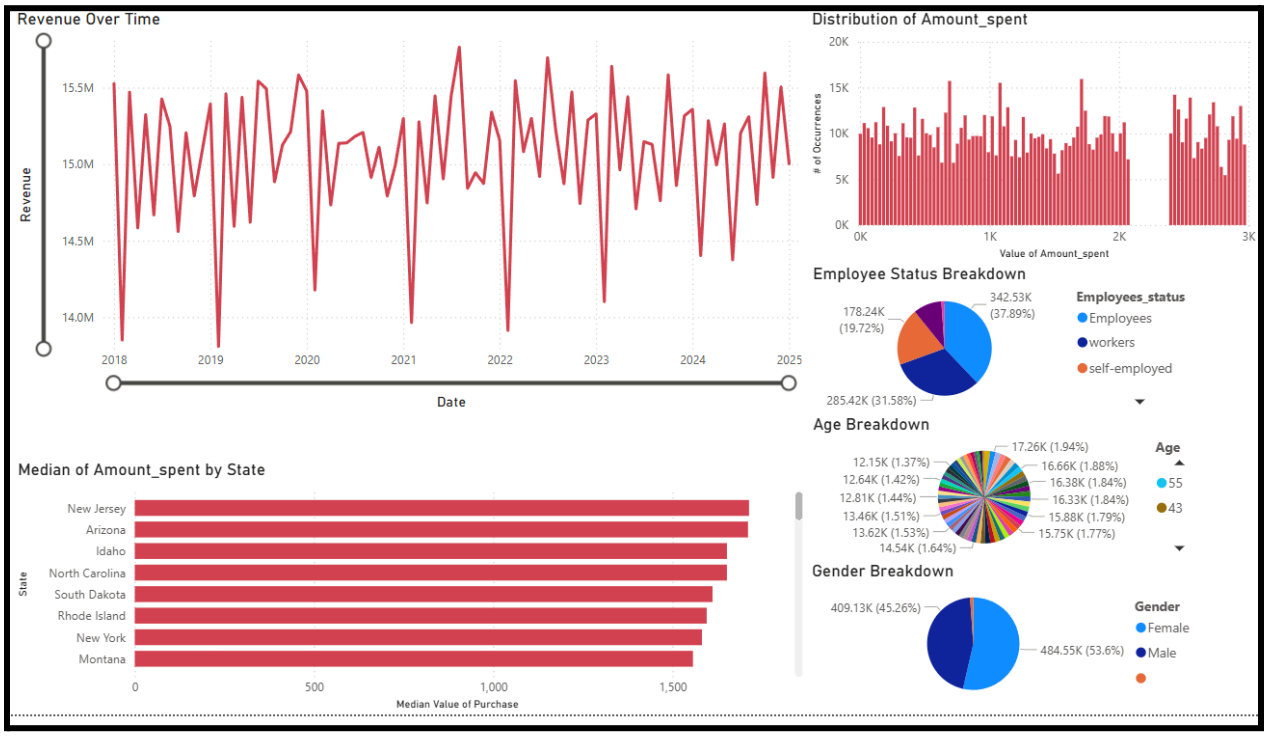
From column impact, we see that the most impactful column was actually gender at 30.36%. From the residuals and error density plots, we can see the model was very slightly likely to under predict the true value.



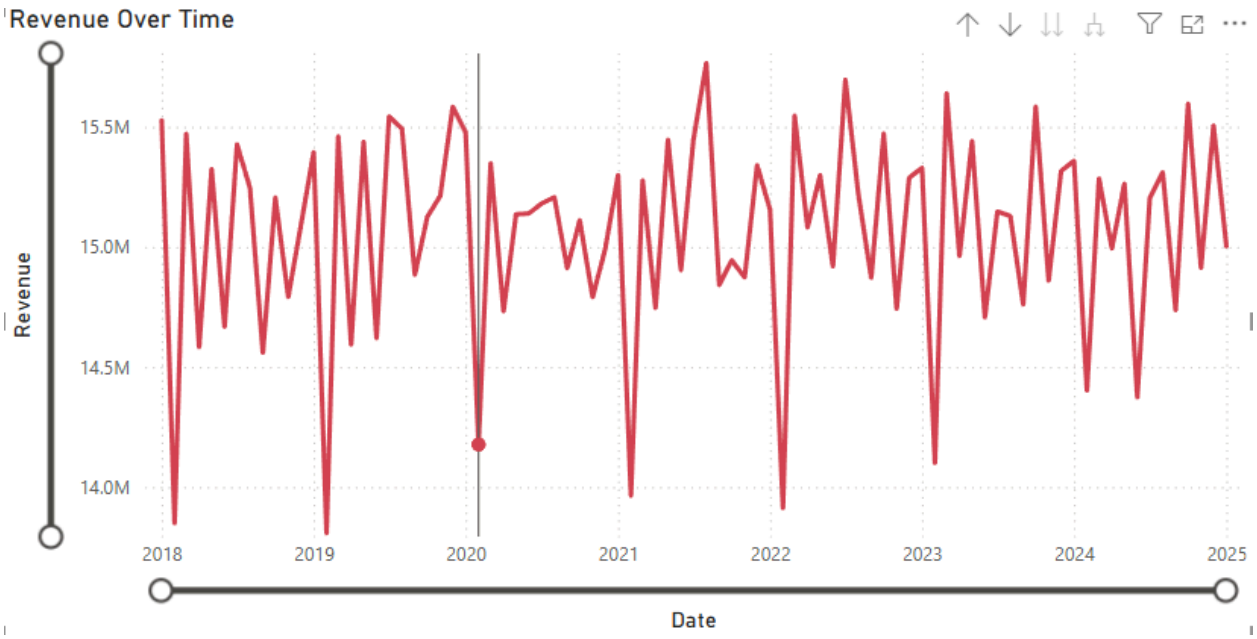
Rounding out the methodology, I designed multiple interactive visualizations to better understand the data. Initially, I tried to use AWS QuickSight which was recently renamed to AWS Quick Suite, but for some reason I literally could not create an AWS account. The page to the left would literally reset with no output every time I tried to confirm a new account.

Since I couldn't get QuickSight to cooperate and it could potentially create additional costs for this project, I decided to proceed with PowerBI as recommended. PowerBI could handle the dataset and the complex interactive visualizations I wanted to create and there is a summary image of the dashboard below. I'll go more in depth with specific insights and plots in the results section of this report.

DASHBOARD SUMMARY:

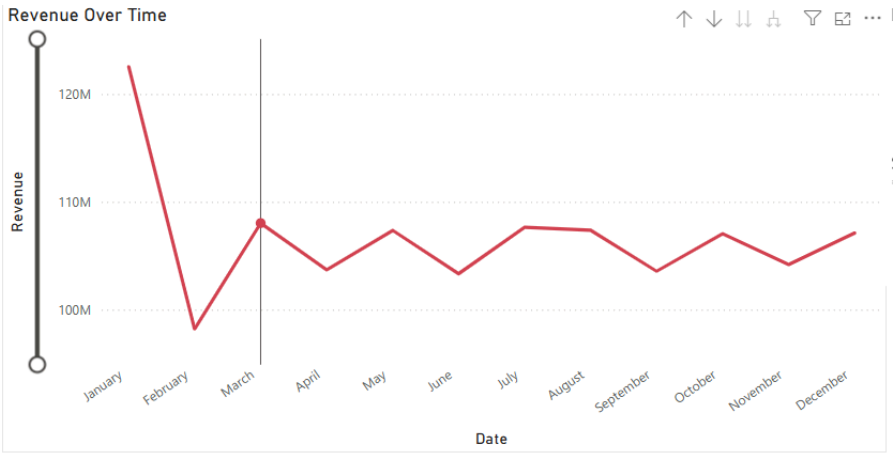


Results:

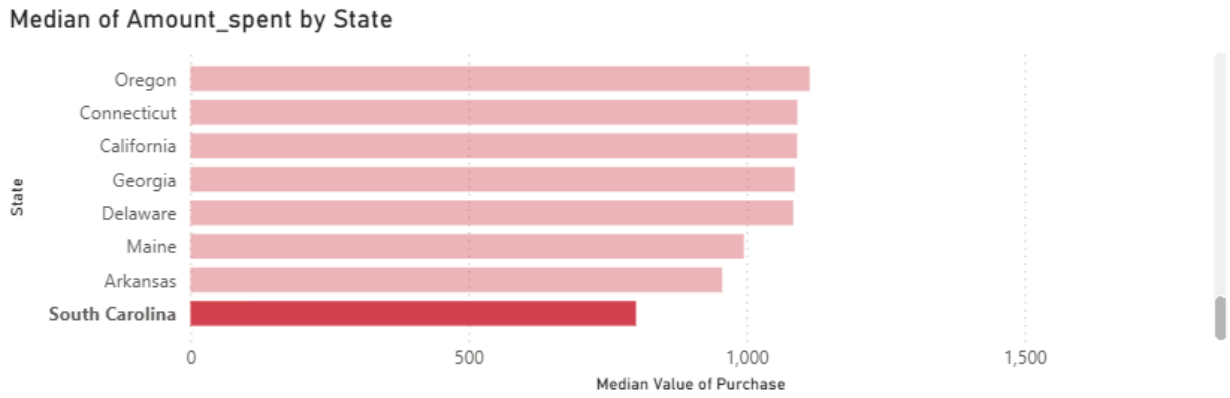


From graphing the Revenue generated by month & year, we see largely consistent sinusoidal variation where it's very rare for the revenue to increase through multiple sequential years. However, we also see a consistent drop in revenue every February of every year (as

designated by the dot and the vertical gray line). According to the table aggregations from step 3.2, we know the average Amount_spent per transaction for each month is generally the same. Therefore, the only possible explanation for the consistent dip in February is low sales where less transactions are being made. This phenomena can be further seen by drilling down to the month:

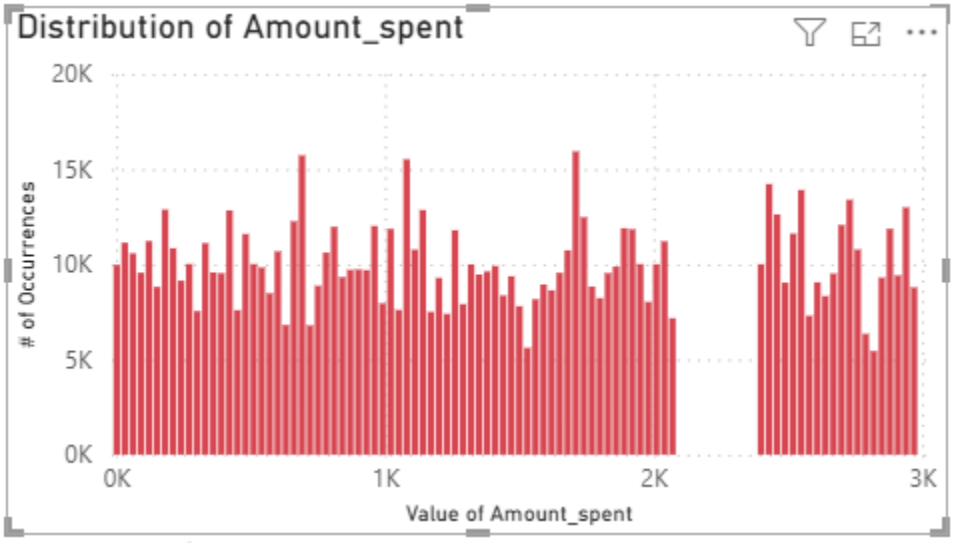


February clearly has the lowest total revenue and January has the highest because as we discussed previously, 2025 didn't have measurements for the entire year and this plot essentially confirms the 2025 measurements only go up to January. Moving onto the next figure, I've graphed the 50 states by the median transaction amount similar to previous aggregations. However, the bar graph allows us to scroll down to see the medians of all 50 states and we can easily view the states with the lowest medians.

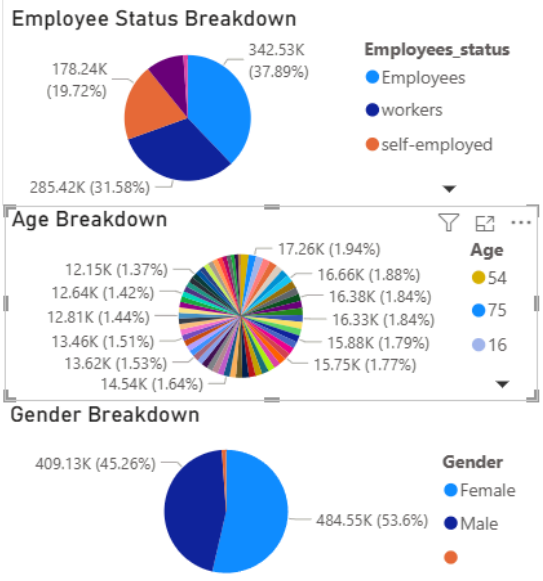


South Carolina has the lowest median value of purchase under half of the median purchase value of New Jersey. By selecting South Carolina and New Jersey we can see the average revenue per month from the previous figure. South Carolina averages about 190k in revenue per month, whereas New Jersey averages about 380k in revenue per month. Overall, there doesn't appear to

be a geographic trend in median Amount_spent by state and the data appears to be set up arbitrarily. For the third figure, I wanted to view the distribution of transaction value in terms of occurrence to see if Amount_spent was skewed in either direction. I created a histogram with 30 dollar ranged bins and the number of occurrences on the y-axis.



We can see from the histogram that the values are not skewed towards either side of the chart. However, there is an obvious gap in values between the 2070 bin and the 2400 bin that appears problematic. After confirming with the original dataset and the SQL queries, there are actually no Amount_spent values within that range. A real world explanation for this phenomena would be a lack of any product that costs between this range, but this is likely just a quirk of the original dataset.



Lastly, I generated three pie charts that communicate the distribution of values in some of the most important categorical columns. Through clicking on specific slices in the pie chart, we can view how customers from that category influenced revenue and Amount_spent. All categories in employee status had virtually no influence on overall revenue and the Amount_spent distribution. The most common age was 54, but as seen

previously, the most common age bracket is still 40-49. After verifying with the pie chart, all age brackets had similar distributions in transaction value, but generally older age groups spent more for individual purchases. Comparing overall revenue between Male and Female, we find that Female contributes more to the monthly revenue than male. Theoretically, this could be caused by the demographic split where Female is 8% larger than Male. However, by looking at the median individual purchase distribution, Female has many outlying bins distributed towards the higher values. Supporting this, from the column impact, we know that Gender actually had the greatest impact at 30% on transaction value followed by Age and State_names. Employee_status only has an impact of 10.3% with other categories like Segment and Payment_methods being as low as 4%. Ultimately, the gender of the customer was the most important with female customers generating significantly more monthly revenue than male customers.

Conclusion:

Through this project, I've deepened my understanding about AWS. I've designed, bug-tested, and implemented a workflow to derive meaningful insights from a large dataset. In working with the Online Store Customer Transactions, I've created scripts, aggregated tables, deep learning models, and interactive visualizations ultimately culminating in 4 key insights. First, February consistently creates the least revenue out of all 12 months. Second, Illinois generated the most revenue, but New Jersey actually had the highest median transaction value. Third, gender had the greatest impact on the transaction value with women generating the most revenue for the company. Fourth, the most common age bracket was 40-49 years old with older brackets spending more money per transaction on average. In terms of applying these insights to business action, I could see reducing the product stock due to the lack of popularity in February. Alternatively increasing marketing and store deals in February could help mitigate some of the loss. Since women generate the most revenue and have the greatest impact on the purchase value, I could see fully leaning into marketing towards women as the product seems to already be preferred by women.

Again, in terms of ethics, I'm going to treat this synthetic dataset as if it were a real dataset. While the dataset does contain anonymized transaction IDS, I'd argue there is enough categorical data to tie the transaction_IDs back to real people. Granted, some of this categorical

data was opt out, like gender, employment_status, and age which helps promote privacy. Ultimately, I would say the Online Store Customer Transactions dataset would be great for internal marketing analysis, but I would strongly disagree with selling this data to other companies or making it public.

Overall, I had an engaging experience with this project and I feel like I've learned a lot. I've seen AWS advertisements all of the time and before this project I had very little understanding what AWS was. I now feel very confident in working with AWS in the future.

References:

Primary Dataset

Chalise, N. (2025). *Online store customer transactions (1M rows)* [Data set]. Kaggle. https://www.kaggle.com/datasets/mountboy/online-store-customer-transactions-1m-rows

Kaggle

Kaggle. (n.d.). *Kaggle: Your machine learning and data science community*. https://www.kaggle.com

U.S. Census Bureau

U.S. Census Bureau. (n.d.). *United States Census Bureau*. https://www.census.gov

Outlier Detection in Spark

Advancing Analytics. (2020, September 2). *Identifying outliers in Spark 3.0*. https://www.advancinganalytics.co.uk/blog/2020/9/2/identifying-outliers-in-spark-30

Recommender Systems / Review Data

McAuley, J. (n.d.). *Amazon review data (2018)* [Data set]. University of California, San Diego. https://nijianmo.github.io/amazon/index.html

Debugging & Technical Issues

OpenAI. (2024). *ChatGPT (GPT-4 / GPT-5 family)* [Large language model]. https://www.openai.com/chatgpt